

DATA + STORIES SERIES

Become a DATA WIZARD with R

A Practical Guide to Data Manipulation,
Visualization, and Reporting in R



By: Amr Eleraqi

Free sample

This sample of *Become a Data Wizard with R* holds the opening of the book, the pages that explain how to use it, and the whole of Chapter 1, “Your first session in R,” with its exercises and sources. The solutions to the exercises are in Appendix 1 of the book.

The pages are the printed pages, so the page numbers jump where pages are left out: the contents list shows where everything else is.

The scripts, data files, and exercise sheets that go with the book are free at github.com/aeleraqi/data-wizard-r.

This sample is free to read and to share as it is, without changes. Copyright © 2026 Amr Eleraqi. All rights reserved. The code examples may be used freely in your own work.

Become a Data Wizard with R

A Practical Guide to Data Manipulation,
Visualization, and Reporting in R

Amr Eleraqi

Become a Data Wizard with R: A Practical Guide to Data Manipulation, Visualization, and Reporting in R

By Amr Eleraqi

This book is part of the Data + Stories series, dedicated to helping professionals transform data into compelling narratives.

Imprint: Independently published

Copyright © 2026 Amr Eleraqi. All rights reserved. No part of this book may be reproduced without the written permission of the author, except for short quotations in reviews and the code examples, which readers may use freely in their own work.

First edition, 2026.

ISBN 9798160281216 (paperback)

ISBN 9798160281926 (hardcover)

Parts of the text of this book were prepared with the help of AI tools. The author reviewed all of it, and every fact in it was checked against the sources listed at the end of each chapter.

The datasets used in the examples are published by their authors under open licenses: gapminder (CC0), palmerpenguins (CC0), nycflights13 (CC0), and the datasets that come with R. The code output was produced with R 4.3.3 and the package versions listed in “Before you begin.”

Names of software and companies are trademarks of their owners. Facts that change quickly, such as versions, menus, and download pages, are dated October 2026; check them before you rely on them.

Contents

Five rows short	ix
Start here	xiii
The road map	xix
Before you begin	xxvii
The ideas in pictures	xxxi
PART ONE: GETTING STARTED	1
1 Your first session in R	3
2 How R sees data	25
3 Getting data in and out	49
PART TWO: PREPARING DATA	75
4 Asking questions of a table	77
5 Cleaning data	101
6 Reshaping and joining tables	127
PART THREE: UNDERSTANDING AND SHOWING	153
7 Describing data with numbers	155
8 Building charts with ggplot2	179
9 Honest charts, ready to publish	205
10 From question to report	231
Appendix 1 Solutions to the exercises	257
Appendix 2 The reference card	287
Appendix 3 Text, encodings, and languages other than English	297
Appendix 4 A dated guide to tools and further reading	317
Appendix 5 Glossary	325
Appendix 6 Exercises and datasets	341
Appendix 7 A guide for instructors: ten sessions	347
References	365
Index	373
About the author	377

Five rows short

In January 2010 the economists Carmen Reinhart and Kenneth Rogoff released a paper called “Growth in a Time of Debt” through the National Bureau of Economic Research in the United States [1], and in May of that year a shorter version appeared in the *American Economic Review* [2]. It drew on a new historical dataset of government debt that covered 44 countries over about 200 years [2]. One of its comparisons used 20 advanced economies, from Australia to the United States, between 1946 and 2009 [2]. Each country’s years were sorted into four groups by the size of government debt relative to the economy (below 30% of GDP, 30 to 60%, 60 to 90%, and above 90%), and growth was compared across the groups [2, 3]. The authors summed up their results in one sentence: “median growth rates for countries with public debt over roughly 90 percent of GDP are about one percent lower than otherwise; average (mean) growth rates are several percent lower” [2]. In the published figures, average growth in the group above 90% was -0.1% a year [3].

The paper was read far outside economics. It was the only evidence on the consequences of high public debt for growth cited in the budget plan for 2013 that the Republican congressman Paul Ryan proposed and the US House of Representatives passed [3]. In Europe, George Osborne, the United Kingdom’s Chancellor of the Exchequer, and Olli Rehn, the leading economic official of the European Commission, frequently cited it as a significant influence on their thinking [3].

In 2013 Thomas Herndon, a third-year graduate student in economics at the University of Massachusetts Amherst, tried to reproduce the paper’s results for an econometrics project [3, 4]. He could not do it from the published documents, so he asked Reinhart and Rogoff for help, and on 4 April 2013 they sent him the working spreadsheet behind the paper [3, 4]. Herndon told *The Chronicle of Higher Education* that the result had struck him as implausible from the start: “I just thought it was counterintuitive when I first saw their claim” [4].

In the spreadsheet, the formulas that computed the means and medians for the period from 1946 took rows 30 to 44 of the sheet instead of rows 30 to 49 [3]. The rows they missed held the figures for Australia, Austria, Belgium, Canada, and Denmark, the first five of the 20 countries in alphabetical order, so those countries were left out of every mean and median in the comparison [3]. A cell shows the result of its formula and not the range behind it, so an average of 15 rows looks the same as an average of 20 until someone clicks on the cell and reads the range.

←
Rows 30 to 49 are 20
rows. Rows 30 to 44
are 15.

Herndon wrote up what he had found with Michael Ash and Robert Pollin, two economists in his department, and the three posted their critique as a working paper on 15 April 2013 [3]. The spreadsheet error was one of three problems they reported. The paper had also left out early postwar years for which data existed, 1946 to 1950 for Australia and Canada and 1946 to 1949 for New Zealand, all of them years in the group above 90% [3]. And it had given every country the same weight in the averages, however many years the country had spent in a group. Without its first four postwar years, New Zealand contributed a single year to the highest group, 1951, with growth of -7.6% , and that one year counted for as much as the 19 years in which the United Kingdom's debt had been above 90% [3]. With all three problems corrected, average growth in the group above 90% came to 2.2% , not -0.1% [3]. Their own accounting also showed how much the error did by itself. Left in place while the other problems were fixed, it lowered the figure from 2.2% to 1.9% ; it took all three problems together, with a transcription error besides, to bring it down to -0.1% [3].

Reinhart and Rogoff answered in a statement dated 18 April 2013. They wrote that Herndon, Ash, and Pollin “accurately point out the coding error that omits several countries from the averages in figure 2,” and added: “It is sobering that such an error slipped into one of our papers despite our best efforts to be consistently careful” [5]. They rejected the other two criticisms, calling their way of weighting the countries “perfectly natural to us, and hardly unconventional,” and they held that the median growth rate they had reported was “the right order of magnitude” [5]. The critique appeared in the *Cambridge Journal of Economics* in March 2014 [3].

The lesson for an analyst is that a calculation inside a spreadsheet cell can stand unchecked for years, until someone else gets the

working file and repeats every step. The published average had stood for almost three years, and the critique was posted 11 days after the authors sent their file to a reader who wanted to repeat their work.

That is why this book teaches analysis in R code. A script records each step as a line of text (which file was read, which rows were kept, what was averaged, and in what order), so handing over the script with the data hands over the whole calculation. A colleague, or you a year later, can run it from the top and trace each number back to the line that produced it. A wrong step in a script is as visible as a right one, so a reader can find it and run the corrected script again. The book is built toward that kind of work: Chapter 1 sets up the tools, and Chapter 10 ends with a report that anyone who receives it can rerun.

Sources for this opening

1. Reinhart, C. M., and Rogoff, K. S. (2010, January). Growth in a time of debt. *NBER Working Paper* 15639. nber.org/papers/w15639
2. Reinhart, C. M., and Rogoff, K. S. (2010). Growth in a time of debt. *American Economic Review*, 100(2), 573–578. doi:10.1257/aer.100.2.573. Text read in the copy at dash.harvard.edu.
3. Herndon, T., Ash, M., and Pollin, R. (2014). Does high public debt consistently stifle economic growth? A critique of Reinhart and Rogoff. *Cambridge Journal of Economics*, 38(2), 257–279. doi:10.1093/cje/bet075. First posted as PERI Working Paper 322, Political Economy Research Institute, University of Massachusetts Amherst, 15 April 2013. Acknowledgments (for the dates of 4 and 15 April 2013), the introduction, the sections on the data exclusions, the coding error (with note 9) and the weighting, and Tables 1 and 5.
4. Monaghan, P. (2013, 24 April). “They said at first that they hadn’t made a spreadsheet error, when they had.” Interview with Thomas Herndon. *The Chronicle of Higher Education*. chronicle.com/article/they-said-at-first-that-they-hadnt-made-a-spreadsheet-error-when-they-had
5. Reinhart, C., and Rogoff, K. (2013, 18 April). *Response to Herndon, Ash and Pollin*. rogoff.scholars.harvard.edu/sites/g/files/omnuum5901/files/rogoff/files/response_to_herndon_ash_and_pollin.pdf

All web addresses were checked on 4 October 2026.

Start here

Who this book is for

This book is for people who already work with data in spreadsheets and want to do the same work in code. You may work in business, research, or government; you sort and total tables and build pivot tables and charts, but you have never written a program. We assume no more than that, and no statistics beyond percentages and averages. The examples come from many countries and subjects, from life expectancy in 142 countries to flights out of New York, and every dataset ships with R or with a package that the chapter names.

The book is written for reading alone, at your own pace, with R open beside it. Each chapter teaches one part of the work and ends with exercises that stand on their own, so there is no project that runs through the book and you can stop after any chapter with something finished. If you teach, Appendix 7 lays the ten chapters out as ten sessions. If you have used R before, “The road map” on the following pages says which chapters you can skim and how to check that you can.

By the end you will be able to take a raw table, import it, clean it, summarize it, chart it, and hand over a report that someone else can rerun. In practice that means you will be able to:

1. read CSV files and Excel sheets with the column types you intend, and write your results back out;
2. fix names, types, missing values, misspelled categories, dates, and units, each with a line of code that records the decision;
3. pick out the rows and columns you need, compute new columns, and summarize them group by group;
4. reshape a table and join it to another without losing or copying rows;
5. describe a column with numbers that suit its shape, and compare groups fairly;

6. draw charts that readers will read the way the data deserve, and save them for print and screen;
7. write a Quarto document that holds the question, the code, and the answer in one file.

The book stops at description. Statistical inference and models, maps and interactive graphics, databases, and writing your own functions are beyond it, and Appendix 4 lists free books that pick up where it stops.

How the book is organized

The front matter you are reading has four short parts after the opening. “Start here” describes the book and how to read it. “The road map” draws the route through the ten chapters and marks the few places where you have a choice to make. “Before you begin” lists what you need, gives the versions of R and its packages that produced the output printed in this book, and says where to find help. “The ideas in pictures” draws seven ideas that the first four chapters depend on (a table, a script, an object, a function, a package, a type, and the pipe), so that the words are familiar before you meet them in code.

The ten chapters fall into three parts. Part One, *Getting started*, covers the tools and the raw material: Chapter 1 installs R and RStudio and teaches the first habits, Chapter 2 explains how R stores values and why their types matter, and Chapter 3 moves tables between R and files. Part Two, *Preparing data*, is the work that comes before any finding: Chapter 4 asks questions of a table with a few verbs and the pipe, Chapter 5 cleans a messy table, and Chapter 6 reshapes tables and joins them. Part Three, *Understanding and showing*, turns a clean table into something a reader can use. Chapter 7 describes data with numbers, Chapter 8 builds charts with `ggplot2`, and Chapter 9 makes those charts honest and ready to publish; Chapter 10 then brings everything together in a report that can be rerun.

Seven appendices follow. Appendix 1 holds the solutions to every exercise, and Appendix 2 collects the reference cards from the end of each chapter into one list of functions. Appendix 3 deals with text, encodings, and languages other than English. Appendix 4 is a dated guide to tools and further reading, Appendix 5 a glossary,

Appendix 6 a list of the exercises and datasets with their page numbers, and Appendix 7 the guide for instructors. The references and an index close the book.

How to read a chapter

Every chapter is built the same way, so after the first one you will know where to look. It opens with its number, its title, a pictogram, and a one-line promise of what the chapter will give you. Below them sits the chapter card, a shaded panel with a heavy rule along its top. Its five entries, each with a small icon of its own, give *The question* the chapter answers, *What you need* (earlier chapters and installed packages), *What you will be able to do* afterward, the *Time* it takes with the exercises, and the *Data* it uses. Read the card first and check the second entry, because a chapter that needs a package you have not installed will stop at the line that loads it.

Then comes a case: something that happened, with real people, dates, and sources, told in a few paragraphs and followed by its lesson in one sentence. The cases are there because each skill in the book exists to prevent or to find a particular kind of error, and a real error shows what is at stake better than a rule does. Every case is told once in the book; later chapters refer to it in a line.

The sections that follow explain an idea, show the code, show what R printed, and say what to notice in the output. You will learn most by running every chunk yourself as you read and comparing your output with the page. Short boxes labeled *Try it*, with a thick rule down their left side and an icon of a screen with a prompt, ask you to do something at the keyboard straight away, usually a variation on the code above it. Their answers are not printed, because the answer is on your screen and the point is to try.

Two kinds of note are written by hand, in the narrow column to the right of the text. Margin notes stand beside the paragraph they belong to, each with a small curved arrow above it; they check a number in the text, show how to read a symbol aloud, or point to another chapter, and there are at most eight in a chapter. Sticky notes are small square notes set at a slight tilt, with a strip of tape across the top, and are printed in yellow where the page is in color. Each holds one habit or keystroke worth keeping, and there are at most three to a chapter. A *Code anatomy* box, drawn with a dashed

Type the code yourself instead of pasting it. Your fingers learn where the commas and brackets go.

←
A margin note sits beside the paragraph it belongs to, like this one.

border and marked with a magnifying glass, takes one short piece of code, puts a numbered circle on each of its parts in the order you read them, and explains each part in a handwritten line. None of these notes carries anything you need to follow the text, so you can skip them on a first reading and come back to them later.

Three kinds of box appear inside the sections. *Common mistakes*, once in each chapter, framed by a thin line and marked with a warning triangle, lists the errors that beginners most often make with that chapter’s tools, what R says when they happen, and the fix. Read it before the exercises, because most of the errors you will meet in them are on that list. A *Worked example*, on a shaded panel without a frame and marked with a calculator, does a calculation with invented numbers when no real dataset has the numbers the explanation needs, and so it always begins “Suppose” and never names a person or a company. A box headed *As of October 2026*, set between two thin rules and marked with a calendar, holds facts that change quickly, such as version numbers, menu names, release dates, and prices. Those facts were checked when the book went to press; if your screen disagrees with one of them, trust your screen and look up the current details.

Each chapter ends in the same order. The *Exercises*, from four to eight of them, each labeled with its number beside a pencil, use real data and ask questions the text has not answered; every one was solved and checked in R, and the solutions are in Appendix 1. Try each exercise before you look, because working it out from a blank script is where most of the learning happens. After the exercises, the box *For your reference card*, framed with rounded corners, lists the functions the chapter introduced, one line each, and Appendix 2 collects all ten cards. The chapter closes with its numbered sources, and a number in square brackets in the text, written here as [n], points to the entry with that number.

Conventions

Code appears in a monospaced typeface. In the text, function names carry parentheses, as in `round()`, while package names, object names, and column names do not, as in `dplyr`, `gapminder`, and `lifeExp`. Blocks of code sit on a shaded background, and what

R prints in reply appears below the block, with every line of output starting with `#>`.

```
# The mean of the 100 yearly flows of the Nile, rounded
round(mean(Nile), digits = 1)
```

```
#> [1] 919.4
```

The first line of the block, which starts with `#`, is a comment, a note for people, which R ignores. The second is the code, and the line below the shaded block is R's answer. In RStudio's console the answer appears without the `#>`, as `[1] 919.4`. The book marks output this way so that you can tell it from code at a glance. Because R treats everything after a `#` as a comment, a block copied from the e-book with its output lines still runs.

 *R itself prints [1]; the #> was added by this book.*

Keys are written with a plus sign between keys pressed together. Where Windows and macOS use different keys for the same job, the book gives both with a slash, Windows first. Ctrl/Cmd + Enter means Ctrl + Enter on Windows and Cmd + Enter on a Mac, and Alt/Option + minus means Alt + minus on Windows and Option + minus on a Mac. Where a shortcut differs in more than one key, both versions are written out. Menu commands are written with an angle bracket between the menus you open, as in File > New File > R Script.

Lines of code are at most 72 characters long, so a longer instruction is broken over several lines. R reads them as one instruction as long as a parenthesis is still open or the line ends with an operator such as `|>`.

Exercises, figures, and tables are numbered by chapter: Exercise 4.1 is the first exercise in Chapter 4, and Figure 8.2 is the second figure in Chapter 8. The appendices are numbered from 1 to 7. Spelling is American, so the book and its code write `summarize()` and *color*, and numbers in the text use commas between thousands (12,763) while numbers in code and output do not (12763).

Companion files

The book is complete on paper: every dataset comes with R or with a package that Chapter 1 shows you how to install, and Chapter 3 has you write its own files from R before you read them back. For

readers who want them, one web address holds everything else that goes with the book: the scripts of every chapter, the data files of Chapters 3 and 10, the finished report of Chapter 10, and the exercises of every chapter as sheets to print on A4 paper.

github.com/aeleraqi/data-wizard-r

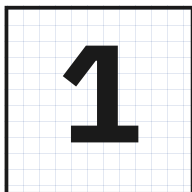
You can open that address in a browser and download the files from there, or let R fetch them for you. Run these three lines in the console of the project you set up in Chapter 1, and R downloads the files and unpacks them into a new folder inside the project:

```
url <- "https://github.com/aeleraqi/data-wizard-r/archive/main.zip"
download.file(url, "companion.zip", mode = "wb")
unzip("companion.zip")
```

Every companion file carries the header “Become a Data Wizard with R”. Use the scripts to check your typing when a chunk refuses to run, after you have looked for the usual causes that Chapter 1 lists. The data files are copies of the files you write yourself in Chapter 3; put them in the `data` folder of your project, as that chapter describes, and the code will work unchanged.

PART ONE

Getting started



CHAPTER 1

Your first session in R

*Install the tools, write a script that will run again tomorrow,
and learn to read what R says back.*

THIS CHAPTER

THE QUESTION

How do I get R running on my computer, give it instructions, and save those instructions so that I, or a colleague, can run them again?

WHAT YOU NEED

A Windows or macOS computer on which you are allowed to install programs, or a web browser and a Posit Cloud account. No programming experience.

WHAT YOU WILL BE ABLE TO DO

Install R, RStudio, and every package this book uses; set up a project; write, run, and save a script; store results in objects; call functions with arguments; find help; and read an error message.

TIME

About three hours, including the installation and the exercises.

DATA

Nile, a series of 100 river flows that comes with R, and a few numbers you type yourself.

In 1990 Ross Ihaka joined the Department of Statistics at the University of Auckland in New Zealand, and in the same year Robert Gentleman, a statistician from the University of Waterloo in Canada, spent a sabbatical in the department [1]. The two shared an interest in statistical computing and, as Ihaka put it, “saw a

common need for a better software environment in our Macintosh teaching laboratory” [2]. No commercial program suited them, so they began to experiment with writing one of their own [2], and when Gentleman joined the department in 1992, they decided to develop enough of a language to teach the introductory statistics courses at Auckland [1].

Both men knew S, a language for data analysis from Bell Laboratories, and they wanted a program driven by typed commands, so they gave theirs a syntax that looks like S [2, 3]. Its inner workings came from Scheme, a language from computer science [3]. They called the language R, partly after their own first names and partly as a play on the name S [3], and in August 1993 they made copies available and announced R in a short message to the s-news mailing list [2].

Some of the people who downloaded R sent comments. “The most persistent of these was Martin Mächler of ETH Zurich,” Ihaka later wrote, and Mächler urged the two to release the source code as free software [2]. They agreed, and in June 1995 the source code became available under the Free Software Foundation’s GNU General Public License [2]. In September 1996 Ihaka and Gentleman described their design in the *Journal of Computational and Graphical Statistics*, writing that they had sought “to combine what we felt were useful features from two existing computer languages” [4].

In mid-1997 a larger core group was given the right to change the source code [2, 3], and on 29 February 2000 Peter Dalgaard announced that R 1.0.0 had been released [5]. Looking back in 2022, Ihaka told the *New Zealand Herald* that the two had briefly thought of selling R: “we thought we might be successful enough to sell 10 copies, [but] who the heck cares about that” [6].



August 1993 to
February 2000: six and
a half years from
announcement to
version 1.0.0.

The lesson for an analyst is that R was built as a language you type, so an analysis done in R is a set of written instructions that a colleague can read, check, and run again. A record of that kind is what makes a mistake like the spreadsheet error described in the opening of this book easier to find.

Why an analyst writes code

In a spreadsheet, most work is done with the mouse: you sort, hide rows, copy a formula down, and build a pivot table, and the sheet shows the result of each step but not the step itself. The formulas live inside cells, where a range that stops one row short looks exactly like one that does not. When next month's figures arrive, you repeat the clicks and hope that you remember all of them, in the right order.

In R you write each step as a line of text: take this table, keep these rows, compute this average. The lines are kept in a script, a plain text file that sits in the same folder as your data. Running the script repeats the whole analysis in the same order, on this month's file or the next, and a colleague who opens it can see every decision you made.

You will install two programs. R is the language and the program that runs it; it is free software, maintained by the R Core Team, and it runs on Windows, macOS, and Unix-like systems such as Linux [3]. RStudio, made by the company Posit, is the program in which you write and run R code, and Posit describes it as a free, open-source development environment for R [7]. RStudio needs R to run your code, so R goes on first.

Installing R and RStudio

R is distributed through CRAN, the Comprehensive R Archive Network, a collection of sites around the world that carry identical copies of R, its documentation, and the packages other people have contributed [3]. The steps differ a little between Windows and macOS, and file names change several times a year, so the box after the steps gives the details as of October 2026.

To install R on Windows:

1. Go to cran.r-project.org, choose Download R for Windows, and then choose base, which the page describes as what you want “to install R for the first time” [8].
2. Download the installer, a file whose name ends in `.exe`, open it, and accept the suggested answer at each step.

To install R on macOS:

1. Find out which kind of processor your Mac has. Open the Apple menu and choose About This Mac: a Mac with Apple silicon lists a Chip, and an Intel Mac lists a Processor [9].
2. Go to cran.r-project.org, choose Download R for macOS, and download the installer for your processor, the file ending in `arm64.pkg` for Apple silicon or the one ending in `x86_64.pkg` for Intel [8].
3. Open the downloaded file and follow the installer to the end.

With R in place, download RStudio Desktop from Posit’s site. On Windows, run the installer and follow its prompts; on macOS, open the disk image that ends in `.dmg`, drag the RStudio icon to the Applications folder, and eject the disk image, as Posit’s installation guide describes [10]. From now on, open RStudio rather than R, because RStudio starts R for you.

AS OF OCTOBER 2026

The current version of R is 4.6.1, released on 24 June 2026, and version 4.6.2 is scheduled for 29 October 2026 [11]. CRAN offers R 4.6.1 for Windows as a 64-bit installer of 88 megabytes that needs no extra software on Windows 10 and later. For macOS it offers `R-4.6.1-arm64.pkg`, for Apple silicon Macs with macOS 14 (Sonoma) or later, and `R-4.6.1-x86_64.pkg`, for Intel Macs with macOS 11 (Big Sur) or later [8]. RStudio Desktop 2026.09.0 runs on Windows 11 and macOS 14 or later; its installers are listed in Posit’s user guide at docs.posit.co/ide/user, to which the old address, posit.co/download/rstudio-desktop, redirects [12]. If your computer runs an older system, use Posit Cloud, described below. This book needs R 4.1.0 or later, and its printed output was produced with R 4.3.3.

TRY IT

Open RStudio. In the pane labeled Console, type `R.version.string` and press Enter. R answers with its version number and release date; check that it is the version you installed.

If you cannot install programs, Posit Cloud runs RStudio inside a web browser, with R already set up on Posit’s computers; Posit describes it as “RStudio and Jupyter Notebooks in your browser” [13]. You sign up on its website, create a project with the New Project button, and work in the same panes as in RStudio

Desktop. Each Posit Cloud project keeps its own packages [13], so create one project for this book and run the installation command from the Packages section inside it. Some keys belong to the browser: Ctrl + Shift + N, for example, does not open a new script in Chrome on Windows [12], so use the menus when a shortcut does something else.

 AS OF OCTOBER 2026

Posit Cloud is at posit.cloud. Posit describes its free tier as “a fully functional environment, available to anyone, with no expiration date,” and offers paid plans with more computing power [13]. Check the plans page for the limits of the free tier.

Finding your way around RStudio

RStudio divides its window into four panes, drawn in Figure 1.1. By default the source editor sits at the top left, the console at the bottom left, the environment pane at the top right, and the output pane at the bottom right [12]. The source editor opens when you open or create a file, so until you do, you may see only three panes, with the console filling the left side.

The console is where R runs code. It shows a prompt, `>`, and waits. Type `1 + 1` after the prompt, press Enter, and R prints the answer, `2`, on the next line and shows a new prompt. Esc stops a command that is taking too long [12]. Commands typed straight into the console are hard to find again once you close RStudio, which is why the work itself belongs in the source editor.

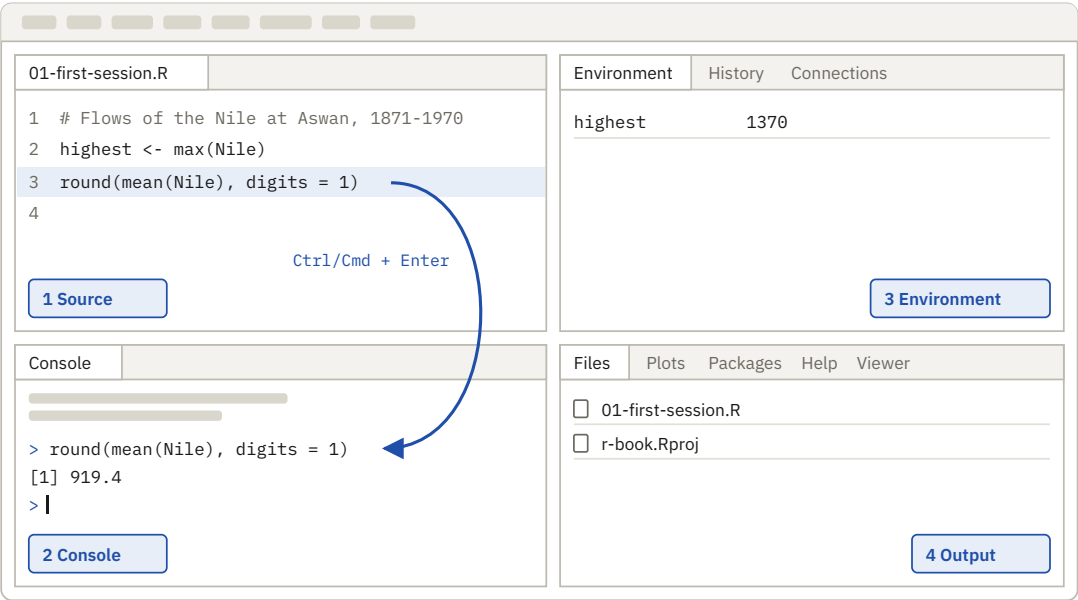


Figure 1.1. The four panes of RStudio in their default positions, drawn by hand. A line run from the script in the source editor (1) appears in the console (2) with its result; the environment pane (3) lists the objects you create; the output pane (4) holds the files, plots, packages, and help tabs.

The environment pane lists every object you have created in the current session, with its value or a short description. The output pane has several tabs: Files shows the folder you are working in, Plots shows the charts you make, Packages lists the packages you have installed, and Help shows documentation [12]. You can move the panes around, but this book assumes the default layout.

Projects and scripts

R reads and writes files relative to one folder, called the *working directory*. When a script asks for a file by its name, R looks for it in that folder, and when it saves a chart, the chart lands there [14]. If the working directory is wherever R happened to start, the same script finds its files on one computer and fails on another. An RStudio project solves this: it is an ordinary folder with a small file ending in `.Rproj`, and opening the project starts a new R session with the working directory set to that folder [12]. Keep the scripts and data for one piece of work inside its project folder, refer to files by their names within it, and the work will run wherever the folder

is copied [14]. Chapter 3 returns to file paths when you start reading data.

Create one project for this book now. Choose File > New Project, click New Directory and then New Project, type a name for the folder, such as `r-book`, choose where it should go, and click Create Project [14]. RStudio reopens inside the new project. Next time, open it by double-clicking its `.Rproj` file or with File > Open Project [12]. To check where R is working, run `getwd()`, which prints the path of the working directory.

```
getwd()
```

On Windows the answer looks something like

`C:/Users/you/Documents/r-book`, and on macOS like

`/Users/you/Documents/r-book`, with your own user name and folders.

With the project open, create a script with File > New File > R Script, or with Ctrl/Cmd + Shift + N, and save it right away with Ctrl/Cmd + S under a name that says what it holds, such as `01-first-session.R` [14]. Then type a line, leave the cursor on it, and press Ctrl/Cmd + Enter. RStudio sends the line to the console, R runs it, and the cursor moves to the next statement, so pressing the keys again steps through the script one statement at a time [14]. To run several lines at once, select them first; to run the whole script from top to bottom, press Ctrl/Cmd + Shift + Enter, which shows each line with its result [12].

R ignores everything on a line after a `#`, which lets you leave notes in the script for the people who read it. These *comments* are most useful when they say why a line is there or where a number came from, because the code already says what it does.

Unless you tell it otherwise, RStudio offers to save the objects in memory when you quit, in a file called `.RData`, and loads them the next time, so a result can survive without the code that made it. The authors of *R for Data Science* advise switching this off, so that everything important has to be in your code, and restarting R often, with Session > Restart R, to check that the script still runs from the top [14].

←
Ctrl on Windows, Cmd on macOS, wherever this book writes Ctrl/Cmd.

Ctrl/Cmd + Enter runs the line under the cursor and moves to the next one.

 AS OF OCTOBER 2026

To stop RStudio from saving and reloading your objects, open Tools > Global Options, uncheck “Restore .RData into workspace at startup,” and set “Save workspace to .RData on exit” to “Never” [14]. The shortcut to restart R is Ctrl + Shift + F10 on Windows and Cmd + Shift + 0 on macOS [12]. Menu names move between versions of RStudio; if you cannot find an option, search the options window for it.

R as a calculator

The quickest way to get used to R is to use it as a calculator. It knows the usual operators: `+` and `-`, `*` for multiplication, `/` for division, and `^` for powers. Chapter 4 works with a dataset that has a row for each country every fifth year from 1952 to 2007, and two lines of arithmetic tell you how many years that is. Type them into your script, with their comments, and run them.

```
# Span of the data, in years
2007 - 1952
```

```
#> [1] 55
```

```
# One row every five years, counting the first
(2007 - 1952) / 5 + 1
```

```
#> [1] 12
```

The 1 in square brackets before each answer marks the first value of the result, which becomes useful when a result holds many values. The parentheses in the second line are needed, because R follows the order of operations from school arithmetic: powers first, then multiplication and division, then addition and subtraction, and parentheses change that order wherever you put them [15].

```
2 + 3 * 4
```

```
#> [1] 14
```

```
(2 + 3) * 4
```

```
#> [1] 20
```

R prints some large numbers, mostly round ones, in scientific notation [15]. It writes 5,000,000,000 as `5e+09`, which means 5×10^9 , a 5

followed by nine zeros, and it accepts the same notation when you type a number: `1e9` is one billion.

```
5 * 1e9
```

```
#> [1] 5e+09
```

R also comes with small datasets for practice. One of them, `Nile`, holds the annual flow of the river Nile at Aswan, in Egypt, for every year from 1871 to 1970, in units of 10^8 cubic meters, that is, 100 million cubic meters [16]. Type its name and R prints all 100 values.

```
Nile
```

```
#> Time Series:
#> Start = 1871
#> End = 1970
#> Frequency = 1
#> [1] 1120 1160 963 1210 1160 1160 813 1230 1370 1140 995 935 1110
#> [14] 994 1020 960 1180 799 958 1140 1100 1210 1150 1250 1260 1220
#> [27] 1030 1100 774 840 874 694 940 833 701 916 692 1020 1050
#> [40] 969 831 726 456 824 702 1120 1100 832 764 821 768 845
#> [53] 864 862 698 845 744 796 1040 759 781 865 845 944 984
#> [66] 897 822 1010 771 676 649 846 812 742 801 1040 860 874
#> [79] 848 890 744 749 838 1050 918 986 797 923 975 815 1020
#> [92] 906 901 1170 912 746 919 718 714 740
```

The first four lines describe the series: it starts in 1871, ends in 1970, and has one value a year. The numbers in square brackets count the values. On this page the second line of numbers starts with the 14th value, the flow for 1884; in your console the lines break where the pane's width allows, and the bracketed numbers change to match. The largest flow, 1,370 units, is the ninth value, for 1879, and the smallest, 456 units, is the 43rd, for 1913.

←
 $1871 + 13 = 1884$, so
 value 14 is the flow for
 1884.

If you press Enter before a command is complete, for example after an opening parenthesis without its partner, the console shows `+` in place of the prompt and waits. Type the rest and press Enter, or press Esc to abandon the command [12, 15].

TRY IT

Use the first and last years of the `Nile` series to compute how many years it covers, counting both ends. Add a comment that says where the two years came from, run the line, and check the answer against the number of values R printed.

Asking R yes-or-no questions

Analysis is full of yes-or-no questions: is this value above the threshold, is this row from 2007, is this country on the list. R answers them with comparison operators, which return `TRUE` or `FALSE`.

```
1000 >= 1000
#> [1] TRUE

"Aswan" == "aswan"
#> [1] FALSE
```

The double equals sign asks whether two things are equal. A single equals sign does something else: inside a function's parentheses it gives an argument its value, as you will see later in this chapter, and on a line of its own it stores a value, like `<-`. Comparisons work on text as well as numbers, and they are exact: `"Aswan"` and `"aswan"` differ by one capital letter, so R answers that they are not equal. Text goes inside double quotation marks, which is how R tells the word Aswan from an object with that name.

←
Count the signs: == compares, a lone = assigns.

Table 1.1 lists all six comparison operators, together with the three logical operators that combine their answers.

Table 1.1. Comparison and logical operators.

Operator	Question it asks	Example	Answer
>	greater than?	1370 > 1000	TRUE
<	less than?	456 < 400	FALSE
>=	greater than or equal to?	1000 >= 1000	TRUE
<=	less than or equal to?	919 <= 900	FALSE
==	equal to?	2 + 2 == 4	TRUE
!=	not equal to?	2 + 2 != 4	FALSE
&	are both true?	TRUE & FALSE	FALSE
	is at least one true?	TRUE FALSE	TRUE
!	what is the opposite?	!TRUE	FALSE

The logical operators combine answers: `&` (and) is `TRUE` only when both sides are, `|` (or) when at least one side is, and `!` (not) turns each answer into its opposite. All of these operators work on the

100 flows of Nile at once. `Nile > 1000` asks the question of every year and gives 100 answers, and wrapping it in `sum()` adds the answers up, counting each `TRUE` as 1 and each `FALSE` as 0. The result is a count of the years that pass the test; Chapter 2 explains why that works.

```
# Years with a flow above 1,000 units
sum(Nile > 1000)
```

```
#> [1] 30
```

```
# Years with a flow from 800 to 1,000 units, both ends included
sum(Nile >= 800 & Nile <= 1000)
```

```
#> [1] 44
```

```
# Years that were not above 1,000 units
sum(!(Nile > 1000))
```

```
#> [1] 70
```

The flow was above 1,000 units in 30 of the 100 years and between 800 and 1,000 units in 44. Each side of `&` has to be a complete comparison: `Nile >= 800 & <= 1000` stops with an error, because `<= 1000` on its own does not say what is being compared. Conditions like these are what you will write in Chapter 4 to keep the rows of a table that you need.

←
30 + 70 = 100: every year was either above 1,000 or not.

Keeping results in objects

Typing 1,370 and 456 every time you need them invites mistakes and hides where the numbers came from. Instead you can store a value under a name with `<-`, the assignment arrow, and use the name from then on. Whatever you store this way is called an *object*.

```
highest <- 1370
lowest <- 456
highest - lowest
```

```
#> [1] 914
```

```
highest / lowest
```

```
#> [1] 3.004386
```

The two assignments print nothing: R stores the values, and the objects appear in the environment pane with their values next to

←
Read <- aloud as "gets":
highest gets 1370.

them. When you use a name in a calculation, R uses the value stored under it, and the largest annual flow turns out to be about three times the smallest. In RStudio, Alt + minus on Windows and Option + minus on macOS types the arrow for you [12].

*Alt/Option + minus
types the
assignment arrow,
<-.*

🔍 CODE ANATOMY

1 `flow_ratio` **2** `<-` **3** `highest / lowest`

1 `flow_ratio` is the name of the new object

3 `highest / lowest` is the calculation, done first, using two objects

2 `<-` stores the result of the right-hand side under that name

Assigning to a name that already exists replaces its value without warning, which is one reason to choose names that say what they hold.

A name can contain letters, digits, the period, and the underscore, and it must start with a letter, or with a period that is not followed by a digit. It cannot contain spaces or hyphens, and it cannot be one of R's reserved words, such as `TRUE`, `FALSE`, `NA`, `if`, and `function` [15]. Names are case sensitive, so `highest`, `Highest`, and `HIGHEST` are three different objects. Within those rules, this book follows the tidyverse style guide: lowercase letters, digits, and underscores, with an underscore between words, a style called snake case: `flow_ratio`, `day_one`, `rows_2007` [17]. A name that breaks the rules stops with an error.

```
1874_flow <- 1210
```

```
#> Error: unexpected input in "1874_"
```

R stops at the underscore, because a name cannot begin with a digit; `flow_1874` is a valid name for the same value.

Functions, arguments, and help

Most of the work in R is done by functions. A function takes one or more inputs, called *arguments*, inside parentheses, and returns a result. R comes with a large set of them, and four short ones describe the `Nile` series.

```
length(Nile)
```

```
#> [1] 100
```

```
mean(Nile)
```

```
#> [1] 919.35
```

```
max(Nile)
```

```
#> [1] 1370
```

```
min(Nile)
```

```
#> [1] 456
```

The series has 100 values with a mean of 919.35 units, and the largest and smallest match the ones you read from the printout. Writing `highest <- max(Nile)` instead of typing 1,370 would have been safer, because the number then comes from the data.

Many functions take more than one argument. `round()` rounds a number: its first argument is the number to round, and its second, `digits`, says how many decimal places to keep.

```
round(919.35)
```

```
#> [1] 919
```

```
round(919.35, digits = 1)
```

```
#> [1] 919.4
```

```
round(919.35, 1)
```

```
#> [1] 919.4
```

You can give an argument by name, as in `digits = 1`, or by position, in the order the function expects, so the second and third lines are the same call. Names make code easier to read later, so use them whenever an argument's meaning is not obvious from its position. When you leave an argument out, the function uses its

default value, and `args()` shows the arguments of a function together with their defaults.

```
args(round)
```

```
#> function (x, digits = 0)
#> NULL
```

The first argument of `round()` is called `x`, and `digits` is 0 unless you say otherwise, which is why `round(919.35)` returned a whole number. The `NULL` on the last line is part of how `args()` prints its answer and can be ignored.

🔍 CODE ANATOMY

```
1 round(2 3.14159, 3 digits 4 = 2)
```

1 `round` is the name of the function

2 `3.14159` is the first argument, matched by position to `x`

3 `digits` is the name of the second argument

4 `= 2` gives that argument its value, with a single equals sign

Functions can be nested, one inside another, so that the result of the inner function becomes an argument of the outer one. R works from the inside out.

```
round(mean(Nile), digits = 1)
```

```
#> [1] 919.4
```

R first computes the mean, 919.35, and then rounds it to one decimal place. Nesting more than two or three functions soon becomes hard to read, and Chapter 4 introduces the pipe, which writes the same steps from left to right.

Every function in R has a help page written by its authors, and the pages share a layout. Type a question mark followed by the function's name in the console, and the page opens in the Help tab.

```
?round
```

The page for `round()` also covers a few related functions. Its *Description* says what they do; *Usage* shows how to call them, with each default, as in `round(x, digits = 0)`; *Arguments* explains each argument; *Details* gives the rules and the exceptions; and

Examples, at the bottom, holds code you can copy and run [15]. Help pages are written for people who already know some R, so read Usage, Arguments, and Examples first and come back to the rest when you need it. The page holds a surprise, which Exercise 1.5 asks you to find.

Running `example(round)` runs the code in the Examples section, which is often the fastest way to see what a function does. When you do not know a function's name, two question marks search the help pages of every installed package: `??"standard deviation"` lists the pages whose topics, titles, or concepts match the phrase, allowing for small differences in spelling [15]. For problems the help pages do not answer, search the web for the text of the error message.

TRY IT

Open the help page for `mean()` with `?mean`. Which argument would you use to skip missing values, and what is its default? Then run `example(mean)`.

Packages

R as it is installed is called base R. Everything beyond it comes in packages, bundles of functions, data, and documentation that anyone can write and publish, and as of October 2026 CRAN distributes nearly 23,000 of them [18]. This book uses more than a dozen, most of them from the tidyverse, a collection of packages for data analysis that “share an underlying design philosophy, grammar, and data structures” [19].

Using a package takes two steps. `install.packages()` downloads a package from CRAN and saves it on your computer, once per computer (or once per Posit Cloud project) and again whenever you want a newer version. `library()` loads an installed package into the current session so that its functions and data can be used, in every session that needs it. That is why `library()` lines go at the top of scripts and `install.packages()` lines usually stay out of scripts.

🔍 CODE ANATOMY

① `install.packages("② gapminder")`

③ `library(④ gapminder)`

① `install.packages` downloads and installs: once per computer

② `gapminder`, inside quotation marks, is the package's name

③ `library` loads the package: once in every session

④ `gapminder` is the name again, here without quotation marks

Install every package this book uses now, in one command, while you have a good internet connection. The `c()` function, which Chapter 2 explains, combines the eight names into one vector.

Install a package once per computer, or once per Posit Cloud project. Load it with `library()` in every session.

```
install.packages(c("tidyverse", "writexl", "gapminder",
                  "palmerpenguins", "nycflights13", "datasauRus",
                  "janitor", "colorspace"))
```

Installing `tidyverse` installs its nine core packages, `ggplot2`, `dplyr`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, `forcats`, and `lubridate`, together with others such as `readxl`, which reads Excel files [19]. R also installs the packages these depend on, so the installation downloads far more than eight and can take a while [15]. Of the other names, `writexl` writes Excel files (Chapter 3), `janitor` tidies column names (Chapter 5), `colorspace` works with colors and palettes (Chapter 9), and the remaining four hold datasets for the examples and exercises. If R asks whether to install from source a package that needs compilation, answer no: R then installs the ready-made version for your system, which may be slightly older [15]. On Windows, R may also offer to create a personal folder for your packages; answer yes [15].

Even after the installation has finished, asking for the `gapminder` data fails; after `library()`, the same name works.

```
gapminder
```

```
#> Error: object 'gapminder' not found
```

```
library(gapminder)
nrow(gapminder)
```

```
#> [1] 1704
```

`nrow()` counts the rows of a table: 1,704 here, one for each of the 142 countries in each of the 12 years you computed earlier.

←
142 countries × 12
years = 1,704 rows.

`library(tidyverse)` loads the nine core packages at once; this book loads them one at a time, as in `library(dplyr)`, so that each chapter shows where its functions come from. Quarto, a separate program that turns code and text into reports, comes with RStudio from version 2022.07.1 on [12]; Chapter 10 returns to it.

Some examples in this book need recent versions of packages; Chapter 4, for instance, uses a function that first appeared in `dplyr` 1.2.0 [20]. `packageVersion()` tells you which version you have.

```
packageVersion("dplyr")
```

```
#> [1] '1.2.1'
```

To update one package, install it again: `install.packages()` always fetches the current version that CRAN offers for your system [15]. To update everything, run `update.packages()`, which asks about each package that has a newer version, or `update.packages(ask = FALSE)`, which updates them all [15]; the Packages tab in RStudio has an Update button that does the same [12]. Update before you load anything with `library()`, so that R is not using a package while it replaces it. On Windows, when the first or second number of R's own version changes, as from 4.5 to 4.6, the R for Windows FAQ advises installing your packages again, because the new version keeps them in a different folder [21].

Reading error messages

Besides results, R talks back in three ways. An *error* means that R stopped and the command produced no result. A *warning* means that R finished but suspects that the result is not what you wanted. A *message* is information, such as a note that a package prints when it loads.

The most common error says that R cannot find something. A misspelled object name produces “object not found,” and a misspelled function name produces “could not find function.”

```
highest
```

```
#> Error: object 'highest' not found
```

```
Max(Nile)
```

```
#> Error in Max(Nile) : could not find function "Max"
```

The last part of the message says what went wrong, and the part after `Error in`, when there is one, names the call that failed. Both errors here are spelling: R is case sensitive, so `Max` is not `max`, and one missing letter makes `highest` a different name from `highest`. The same messages appear when the spelling is right but the line that creates the object has not been run, or the package that holds the function has not been loaded.



Read an error from the end: the last words say what went wrong.

A second group of errors comes from punctuation. A missing comma or an extra parenthesis leaves R unable to read the line at all, and R says where it gave up. An unclosed quotation mark gives no error: the console shows `+` and waits, so press Esc and correct the line.

```
round(3.14159 digits = 2)
```

```
#> Error: unexpected symbol in "round(3.14159 digits"
```

The quoted text ends where R stopped understanding the line: after the number it expected a comma or a closing parenthesis, and it found `digits` instead.

A third group comes from giving a function the wrong kind of value. Text inside quotation marks is not a number, even when it looks like one, and Chapter 2 explains how R tells the two apart.

```
sqrt("16")
```

```
#> Error in sqrt("16") : non-numeric argument to mathematical function
```

A warning does not stop R. The square root of a negative number is not a real number, so `sqrt()` returns `NaN`, short for “not a number,” and warns you.

```
sqrt(-1)
```

```
#> Warning message:
#> In sqrt(-1) : NaNs produced

#> [1] NaN
```

In the console the warning appears after the result, not before it as on this page. Take warnings seriously, because a result that is partly wrong is harder to spot than an error. When a message makes no sense to you, check the line for the usual causes: capital letters, commas, quotation marks, and parentheses.

COMMON MISTAKES

The first of these costs you work, the next two stop your code, and the last two let it run and give a wrong answer without a word.

1. Working in the console instead of a script: the commands are hard to recover once RStudio closes.
2. Leaving out the quotation marks in `install.packages(tidyverse)` : R stops because it looks for an object called `tidyverse` and finds none. The package name goes in quotes there.
3. Expecting packages to stay loaded after a restart. If a function that worked yesterday gives “could not find function,” the script is probably missing its `library()` line.
4. Writing `=` when you mean `==`: `highest = 1000` does not ask whether `highest` is 1,000; it replaces the value of `highest` with 1,000, and R says nothing.
5. Comparing decimal results with `==`: `0.1 + 0.2 == 0.3` is `FALSE`, because most decimal fractions cannot be stored exactly in a computer [15]. Compare rounded values instead: `round(0.1 + 0.2, digits = 10) == 0.3` is `TRUE`.

Exercises

The solutions are in Appendix 1. Each exercise uses only what this chapter has introduced.

EXERCISE 1.1

The Nile flows are measured in units of 10^8 cubic meters. Convert the largest flow, 1,370 units, to cubic meters, and then to cubic kilometers (one cubic kilometer is 10^9 cubic meters). Do the same for the smallest flow, 456 units. What does the `e+11` in one of R's answers mean?

EXERCISE 1.2

Write down the answer you expect from each line, then run the lines. Which result surprised you, and what does it tell you about the order in which R applies a minus sign and `^`?

```
2 + 3 * 4 ^ 2
(2 + 3) * 4 ^ 2
-2 ^ 2
(-2) ^ 2
```

EXERCISE 1.3

Use `Nile` and `sum()` to count the years with a flow below 700 units, below 700 or above 1,200 units, and exactly 1,000 units. Then use `!` to count the years not below 700 units, and check that this count and the first add up to 100.

EXERCISE 1.4

Write down the value you expect `area` to have at the end, then run the lines. Explain the answer, and say how a spreadsheet would behave if `area` were a cell with a formula multiplying `width` by `height`.

```
width <- 4
height <- 3
area <- width * height
width <- 10
area
```

 EXERCISE 1.5

Use the help page for `round()` to round 1,370 to the nearest hundred, and to round 919.35 to two significant digits with `signif()`, which the same page describes. Then run `round(0.5)`, `round(1.5)`, and `round(2.5)`. What rule is R following, and in which section of the page is it explained?

 EXERCISE 1.6

Each line below stops with an error or gives an answer you should not trust. Run them one at a time, note what R says, and correct each line. The last line was meant to count the years in which the flow was above 1,000 units.

```
round(919.35 1)
mean(Nile, na.rm = true)
max(nile)
sum("Nile" > 1000)
```

 FOR YOUR REFERENCE CARD

<code>x <- value</code>	stores a value under the name <code>x</code>
<code>+ - * / ^</code>	add, subtract, multiply, divide, raise to a power
<code>> < >= <= == !=</code>	compare two values; the answer is <code>TRUE</code> or <code>FALSE</code>
<code>a & b, a b, !a</code>	and, or, not
<code># text</code>	a comment: R ignores the rest of the line
<code>length(x), sum(x), mean(x)</code>	number of values, total, mean
<code>max(x), min(x)</code>	largest and smallest value
<code>round(x, digits = 0)</code>	rounds to a number of decimal places
<code>sqrt(x)</code>	square root
<code>nrow(df)</code>	number of rows in a table
<code>args(f)</code>	shows the arguments of <code>f()</code> and their defaults
<code>?f, example(f)</code>	opens the help page of <code>f()</code> ; runs its examples
<code>??"words"</code>	searches the help pages of installed packages
<code>getwd()</code>	prints the working directory
<code>install.packages("pkg")</code>	installs a package, once per computer or Posit Cloud project
<code>library(pkg)</code>	loads a package, in every session
<code>update.packages()</code>	updates installed packages

<code>packageVersion("pkg")</code>	shows the installed version of a package
<code>Ctrl/Cmd + Enter</code>	runs the current line of a script

Sources for this chapter

1. Ihaka, R. *The R project: A brief history and thoughts about the future*. Slides of a talk, Department of Statistics, University of Auckland, undated. stat.auckland.ac.nz/~ihaka/downloads/Massey.pdf
2. Ihaka, R. (1998). R: Past and future history. Draft of a paper for Interface '98. stat.auckland.ac.nz/~ihaka/downloads/Interface98.pdf
3. Hornik, K., and the R Core Team. *R FAQ: Frequently asked questions on R*, sections 2.1, 2.2, 2.10, and 2.12. Version of 5 April 2023, distributed with R 4.3.3. cran.r-project.org/doc/FAQ/R-FAQ.html
4. Ihaka, R., and Gentleman, R. (1996). R: A language for data analysis and graphics. *Journal of Computational and Graphical Statistics*, 5(3), 299–314. doi:10.1080/10618600.1996.10474713.
5. Dalgaard, P. (2000, 29 February). R-1.0.0 is released. *R-announce mailing list*. stat.ethz.ch/pipermail/r-announce/2000/000127.html
6. Smith, X. (2022, 12 April). Statistics legend Ross Ihaka reflects on his revolutionary software. *New Zealand Herald*. nzherald.co.nz/kahu/statistics-legend-ross-ihaka-reflects-on-his-revolutionary-software/ NSGLPFJMXQK3GVV67UNEFUCHTE
7. Posit. RStudio IDE, product page. posit.co/products/open-source/rstudio
8. CRAN, the Comprehensive R Archive Network: main page, “R for Windows” and its base page, and “R for macOS.” cran.r-project.org, checked on the ETH Zurich mirror, stat.ethz.ch/CRAN.
9. Apple. Mac computers with Apple silicon. *Apple Support*. support.apple.com/en-us/116943
10. Posit. *RStudio Desktop Pro Administration Guide*, “Installation.” docs.posit.co/ide/desktop-pro/latest/getting_started/installation.html
11. The R Foundation. *The R Project for Statistical Computing*, home page and news. r-project.org
12. Posit. *RStudio IDE User Guide*, for RStudio 2026.09.0: home page and downloads; “Pane layout”; “RStudio Projects”; “Quarto projects”; “Keyboard shortcuts.” docs.posit.co/ide/user
13. Posit. Posit Cloud, product page, posit.co/products/cloud/cloud; and *Posit Cloud Documentation*, “Code projects,” docs.posit.co/cloud/guide/projects
14. Wickham, H., Çetinkaya-Rundel, M., and Golemund, G. (2023). *R for Data Science*, 2nd ed., chapter 6, “Workflow: scripts and projects.” r4ds.hadley.nz/workflow-scripts.html
15. R Core Team. Help pages for `Syntax`, `options`, `Reserved`, `make.names`, `round`, `help.search`, `install.packages`, and `update.packages`, R 4.3.3.
16. R Core Team. *Nile: Flow of the river Nile*. Documentation of the `datasets` package. stat.ethz.ch/R-manual/R-devel/library/datasets/html/Nile.html
17. Wickham, H. *The tidyverse style guide*, “Syntax.” style.tidyverse.org/syntax.html
18. CRAN. Available packages. cran.r-project.org/web/packages (22,976 packages on 5 October 2026).
19. The tidyverse team. tidyverse.org and tidyverse.org/packages; and Wickham, H. (2023, 8 March). *tidyverse 2.0.0. Tidyverse blog*. tidyverse.org/blog/2023/03/tidyverse-2-0-0
20. Vaughan, D. (2026, 4 February). `dplyr 1.2.0`. *Tidyverse blog*. tidyverse.org/blog/2026/02/dplyr-1-2-0
21. Ripley, B. D., and Murdoch, D. J., with updates by T. Kalibera. *R for Windows FAQ*, version for R 4.6.1, question 2.8, “What’s the best way to upgrade?” cran.r-project.org/bin/windows/base/rw-FAQ.html

All web addresses were checked on 4 and 5 October 2026.

That was Chapter 1

The book goes on from here to the questions an analyst asks every day: reading spreadsheets and text files into R, filtering, grouping, and summarizing tables, cleaning messy columns and joining tables without losing rows, describing data with the right numbers, drawing charts that tell the truth, and turning an analysis into a report that someone else can run again. Each of the ten chapters opens with a documented case and ends with exercises, and every exercise has a full, tested solution.

GET THE BOOK

Paperback · ISBN 9798160281216 · www.amazon.com/dp/B0HM11PJQV

Hardcover · ISBN 9798160281926 · www.amazon.com/dp/B0HM13QZMV

Kindle edition · on Amazon

COMPANION FILES

github.com/aeleraqi/data-wizard-r

Part of the Data + Stories series. This sample is free to read and to share as it is, without changes. Copyright © 2026 Amr Eleraqi. All rights reserved.